

ui.xterm 全面详细阐述

`ui.xterm` 是 NiceGUI 3.1.0+ 版本新增的终端模拟组件，基于 `xterm.js` 实现，提供前端终端界面渲染能力，支持 ANSI 转义序列、事件监听、自动适配容器大小、子进程输出对接等核心功能。需注意该组件仅提供前端终端界面，无内置底层 Shell，需手动对接后端（如 `pty`、子进程）实现命令执行逻辑，适用于在线终端模拟、命令行工具可视化、日志实时展示等场景。以下从核心特性、使用方法、参数配置、API 详情等方面展开说明。

一、核心特性

- 基于成熟终端库：**底层依赖 `xterm.js`，兼容终端标准特性，支持 UTF-8 编码字符串和字节数据输入。
- ANSI 转义序列支持：**可解析 ANSI 颜色、样式（加粗、下划线）、光标控制等转义码，实现富文本终端输出。
- 完整事件体系：**支持 `data`（用户输入 / 粘贴）、`bell`（终端告警）、`resize`（终端大小变化）等事件，覆盖终端交互全流程。
- 灵活尺寸控制：**支持手动设置行列数，通过 `fit` 方法自动适配容器大小，配合尺寸监听实现动态调整。
- 子进程对接能力：**可绑定异步子进程的标准输出 / 错误流，实时展示命令执行日志，支持换行符自动转换。
- 样式可扩展：**支持通过 `classes / style / props` 自定义终端容器样式，适配不同界面风格。

二、基础使用方法

1. 最简示例：终端输出文本

初始化终端并写入固定文本，设置终端行列数：

```
from nicegui import ui

# 初始化终端：30 列、9 行
terminal = ui.xterm({'cols': 30, 'rows': 9})

# 页面加载时执行一次，写入欢迎文本
ui.timer(0, lambda: terminal.write('Hello NiceGUI!\nwelcome to Xterm Terminal.'), once=True)

ui.run()
```

2. ANSI 转义序列：富文本输出

通过 ANSI 转义码实现文本颜色、样式自定义，`writeln` 方法自动添加换行符：

```
from nicegui import ui

terminal = ui.xterm({'cols': 30, 'rows': 9})

# 普通文本（自动换行）
ui.button('添加普通文本', on_click=lambda: terminal.writeln('This is normal text.'))

# 蓝色文本（ANSI 34m 表示蓝色，0m 重置样式）
ui.button('添加蓝色文本', on_click=lambda: terminal.writeln('\x1b[34mThis text is blue!\x1b[0m'))

# 加粗文本（ANSI 1m 表示加粗）
```

```

ui.button('添加加粗文本', on_click=lambda: terminal.writeLn('\x1b[1mThis text is
bold!\x1b[0m'))
# 绿色背景文本 (ANSI 42m 表示绿色背景)
ui.button('添加绿色背景文本', on_click=lambda: terminal.writeLn('\x1b[42mGreen background
text\x1b[0m'))

ui.run()

```

3. 监听用户输入与终端告警

绑定 `data` 事件接收用户输入 / 粘贴内容, 绑定 `bell` 事件响应终端告警 (如 Ctrl+G 触发):

```

from nicegui import ui

terminal = ui.xterm({'cols': 30, 'rows': 9})

# 处理用户输入: 替换换行符和退格键 (模拟 shell 行为)
def handle_input(e):
    # 替换 \r 为 \n\r (正确换行), 替换 \x7f (退格键) 为光标左移+清除字符
    processed_data = e.data.replace('\r', '\n\r').replace('\x7f', '\x1b[0D\x1b[0K')
    terminal.write(processed_data)

terminal.on_data(handle_input)

# 处理终端告警: 弹出通知 (Ctrl+G 可触发)
terminal.on_bell(lambda: ui.notify('🔔 终端告警被触发 (可能按下了 Ctrl+G)'))

ui.run()

```

4. 自动适配容器大小

通过 `fit` 方法让终端尺寸匹配容器, 结合尺寸监听器实现容器缩放时终端自动调整:

```

from nicegui import ui

# 可缩放容器 (size-60 为固定尺寸, resize 允许手动缩放, overflow-auto 处理溢出)
with ui.card().classes('size-60 resize overflow-auto'):
    terminal = ui.xterm().classes('size-full') # 终端占满容器
    # 尺寸监听器: 容器缩放时触发终端适配
    ui.element('q-resize-observer').on('resize', terminal.fit)

# 显示当前终端行列数 (监听 resize 事件)
size_label = ui.label('Size: 0x0')
terminal.on('resize', lambda e: size_label.set_text(f'Size:
{e.args["cols"]}x{e.args["rows"]}'))

ui.run()

```

5. 对接子进程：实时展示命令输出

通过异步子进程执行命令，将标准输出 / 错误流实时写入终端，支持自动转换换行符：

```
from nicegui import ui
import asyncio

# 初始化终端：启用换行符自动转换 (\n → \r\n)
terminal = ui.xterm({'cols': 30, 'rows': 9, 'convertEol': True})

async def run_subprocess():
    button.disable() # 执行期间禁用按钮
    # 创建异步子进程 (-u 禁用缓冲，确保实时输出)
    process = await asyncio.create_subprocess_exec(
        'python3', '-u', '-c',
        (
            'import time\n'
            'for i in range(5):\n'
            '    print(f"Step {i+1}/5: Processing...")\n'
            '    time.sleep(0.5)\n'
            'print("\x1b[32m✓ All steps completed successfully!\x1b[0m")' # 绿色成功提示
        ),
        stdout=asyncio.subprocess.PIPE, # 捕获标准输出
        stderr=asyncio.subprocess.PIPE # 捕获标准错误
    )

    # 读取流数据并写入终端
    async def write_stream(stream):
        while chunk := await stream.read(128): # 每次读取 128 字节
            terminal.write(chunk) # 支持字节数据直接写入

    # 并发处理输出流和进程等待
    await asyncio.gather(
        write_stream(process.stdout),
        write_stream(process.stderr),
        process.wait()
    )
    button.enable() # 执行完成启用按钮

# 执行子进程的按钮
button = ui.button('运行子进程', on_click=run_subprocess)

ui.run()
```

三、关键参数说明

- `options`：类型为 `dict | None`，xterm.js 原生配置字典，用于设置终端基础属性，默认值为 `None`。核心配置项包括：
 - `cols`：终端列数（字符数），默认根据容器自动计算。
 - `rows`：终端行数（字符数），默认根据容器自动计算。

- `convertEol`: 是否自动将 `\n` 转换为 `\r\n`, 适配终端换行规则, 默认 `False`。
- `cursorBlink`: 光标是否闪烁, 默认 `False`。
- `fontSize`: 字体大小 (如 `14`), 默认继承父元素样式。
- 更多配置可参考 [xterm.js 官方文档](#)。

四、高级功能

1. 手动控制终端尺寸与内容

通过 `get_columns/get_rows` 获取当前行列数, 通过 `write/writeln` 手动写入内容, 结合按钮实现终端控制:

```
from nicegui import ui

terminal = ui.xterm({'cols': 30, 'rows': 5})

# 显示当前尺寸
def show_size():
    cols = terminal.get_columns()
    rows = terminal.get_rows()
    terminal.writeln(f'\nCurrent size: {cols}x{rows}')

# 清空终端 (ANSI 转义码: 清除屏幕+光标归位)
def clear_terminal():
    terminal.write('\x1b[2J\x1b[0f') # 2J 清除屏幕, 0f 光标移至左上角

ui.button('显示当前尺寸', on_click=show_size)
ui.button('清空终端', on_click=clear_terminal)
ui.button('写入多行文本', on_click=lambda: terminal.writeln('Line 1\nLine 2\nLine 3'))

ui.run()
```

2. 对接真实 Shell (基于 pty)

通过 `ptyprocess` 库创建伪终端 (pty), 实现完整 Shell 交互 (需安装 `ptyprocess`: `pip install ptyprocess`):

```
from nicegui import ui
import asyncio
from ptyprocess import PtyProcess
import os

async def run_shell():
    # 创建 pty 进程 (启动 bash 或 sh)
    shell = PtyProcess.spawn([os.environ.get('SHELL', 'bash')])
    terminal = ui.xterm({'cols': 40, 'rows': 10, 'convertEol': True}).classes('h-64')

    # 读取 pty 输出并写入终端
    async def read_pty():
        while shell.isalive():
            try:
```

```

        data = shell.read(1024) # 每次读取 1024 字节
        if data:
            terminal.write(data)
            await asyncio.sleep(0.01)
        except Exception:
            break

# 处理终端输入并写入 pty
def handle_input(e):
    if shell.isalive():
        shell.write(e.data.encode()) # 转换为字节写入 pty

terminal.on_data(handle_input)

# 监听终端 resize 事件，同步调整 pty 尺寸
def handle_resize(e):
    if shell.isalive():
        cols = e.args['cols']
        rows = e.args['rows']
        shell.setwinsize(rows, cols) # pty 尺寸需传入（行数，列数）

terminal.on('resize', handle_resize)

# 启动读取任务
asyncio.create_task(read_pty())

ui.button('启动 shell', on_click=run_shell)
ui.run()

```

3. 自定义终端样式

通过 `Classes / style` 调整终端容器样式，结合 `xterm.js` 配置修改终端内部样式（如字体、背景色）：

```

from nicegui import ui

# 自定义样式的终端：黑色背景、灰色边框、等宽字体
terminal = ui.xterm(
    options={
        'fontSize': 14,
        'fontFamily': 'Consolas, Monaco, monospace', # 等宽字体
        'background': '#000000', # 黑色背景
        'foreground': '#ffffff' # 白色文本
    }
).classes('w-full h-48 border-2 border-gray-400 rounded-md p-2')

# 写入带颜色的系统日志样式文本
terminal.writeln('\x1b[36m[INFO] Starting application...\x1b[0m')
terminal.writeln('\x1b[33m[WARNING] Low memory detected\x1b[0m')
terminal.writeln('\x1b[31m[ERROR] Connection failed\x1b[0m')

ui.run()

```

五、API 详情补充

1. 核心属性

属性名	类型	说明
<code>classes</code>	<code>str</code>	组件 HTML 类名，支持 Tailwind/Quasar 类（如设置尺寸、边框、间距）
<code>client</code>	<code>Client</code>	组件所属的客户端实例
<code>html_id</code>	<code>str</code>	组件 HTML DOM ID（版本 2.16.0+），用于手动定位 DOM 元素
<code>props</code>	<code>Props[Self]</code>	组件原生属性，支持 Quasar 相关配置
<code>style</code>	<code>Style[Self]</code>	组件内联 CSS 样式（如 <code>height: 300px;</code> ）
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定，支持动态切换）

2. 常用方法

方法名	说明	参数	
<code>`write(data: bytes</code>	<code>str)`</code>	向终端写入数据，支持字符串或 UTF-8 编码字节	<code>data</code> ：要写入的内容（字符串或字节）；返回 <code>AwaitableResponse</code> ，可等待写入完成
<code>`writeln(data: bytes</code>	<code>str)`</code>	向终端写入数据并自动添加 <code>\n</code> （换行）	同 <code>write</code> ，返回 <code>AwaitableResponse</code>
<code>fit() -> AwaitableResponse</code>	让终端尺寸适配容器，更新行列数	无参数；返回 <code>AwaitableResponse</code> ，可等待适配完成	
<code>get_columns() -> int</code>	获取当前终端列数（字符数）	无参数，返回整数	
<code>get_rows() -> int</code>	获取当前终端行数（字符数）	无参数，返回整数	
<code>input(data: str,</code> <code>was_user_input: bool = True)</code>	向应用侧输入数据（触发 <code>data</code> 事件）	<code>data</code> ：输入数据； <code>was_user_input</code> ：是否视为用户输入（影响光标 / 选择状态）； 返回 <code>AwaitableResponse</code>	
<code>run_terminal_method(name:</code> <code>str, *args, timeout: float = 1)</code>	调用 xterm.js 原生方法	<code>name</code> ：原生方法名（如 <code>focus</code> 聚焦终端）； <code>args</code> ：方法参数； <code>timeout</code> ：超时时间；返回 <code>AwaitableResponse</code>	
<code>set_visibility(visible: bool)</code>	显示 / 隐藏终端	<code>visible</code> ：True 显示，False 隐藏	

方法名	说明	参数	
<code>tooltip(text: str)</code>	为终端添加悬浮提示	<code>text</code> : 提示文本	
<code>update()</code>	强制更新组件状态到客户端	无参数	

3. 核心事件

事件名	说明	回调参数
<code>data</code>	用户输入或粘贴内容时触发	<code>XtermDataEventArguments</code> 对象, 含 <code>data</code> 属性 (输入字符串)
<code>bell</code>	终端告警被触发时 (如 Ctrl+G)	<code>XtermBellEventArguments</code> 对象, 无额外属性
<code>resize</code>	终端尺寸变化时触发 (自动适配或手动调整)	事件对象 <code>e</code> , <code>e.args</code> 含 <code>cols</code> (列数) 和 <code>rows</code> (行数)
<code>focus</code>	终端获得焦点时触发	通用事件对象
<code>blur</code>	终端失去焦点时触发	通用事件对象

六、注意事项

- 无内置 Shell:** `ui.xterm` 仅提供前端界面, 需手动对接 `pty`、子进程或后端服务实现命令执行, 否则用户输入无实际响应。
- 换行符适配:** 终端默认使用 `\r\n` 作为换行符, 若写入内容换行异常, 需启用 `convertEol: True` 或手动替换换行符。
- pty 兼容性:** Python 原生 `pty` 模块不支持尺寸调整, 推荐使用 `ptyprocess` 库实现完整 `pty` 功能 (如尺寸同步、信号处理)。
- 性能优化:** 高频写入数据时 (如实时日志), 建议批量读取流数据 (如每次 128/1024 字节), 避免频繁调用 `write` 导致性能问题。
- 版本兼容性:** `ui.xterm` 仅支持 NiceGUI 3.1.0+ 版本, `html_id` 属性需 2.16.0+, 使用时需确保框架版本符合要求。
- 样式冲突:** 自定义终端背景色、字体时, 需避免与容器样式冲突 (如容器 `background` 覆盖终端背景), 优先通过 `options` 配置终端内部样式。

七、应用场景

- 在线终端工具:** 为 Web 应用添加在线 Shell 功能, 支持远程命令执行 (需对接后端服务)。
- 日志实时展示:** 展示应用运行日志、系统监控日志, 通过 ANSI 颜色区分日志级别 (`-info/warn/error`)。
- 命令行工具可视化:** 将本地命令行工具 (如 `git`、`docker`) 包装为 Web 界面, 实时展示执行输出。
- 教学演示场景:** 模拟终端操作, 用于编程教学、命令行使用教程等场景。
- 配置脚本执行:** 允许用户输入配置脚本 (如 Python、Shell), 后端执行后实时返回输出结果。

`ui.xterm` 凭借 xterm.js 的强大终端模拟能力和 NiceGUI 的简洁 API，快速实现 Web 端终端交互功能，无需深入前端开发即可获得接近原生终端的使用体验，适配各类终端相关场景的需求。